

TorchDynamo 调研文档

目录

- [什么是 TorchDynamo](#)
 - [TorchDynamo 总览](#)
 - [TorchDynamo 编译过程](#)
- [Eval Frame 原理和执行流程](#)
 - [FrameObject 和 CodeObject](#)
 - [PEP 523](#)
 - [Eval Frame 流程](#)
- [字节码模拟执行](#)
 - [compile 流程](#)
 - [模拟执行流程](#)
 - [子图打断](#)
 - [跳转指令的处理](#)
- [代码生成](#)
 - [子图编译代码生成](#)
 - [Source 管理](#)
 - [Guard 管理](#)
 - [SideEffects 管理](#)
- [References](#)

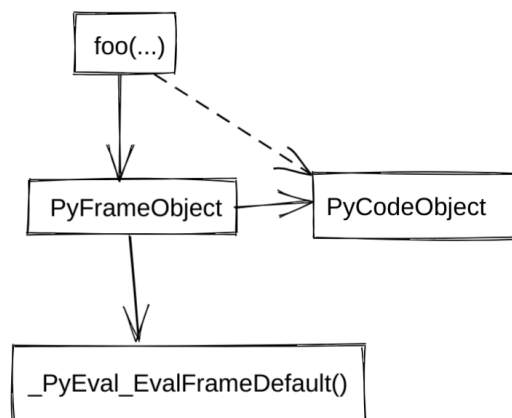
❗ 内容尚为初稿，后续会随着调研深入继续修改

什么是 TorchDynamo

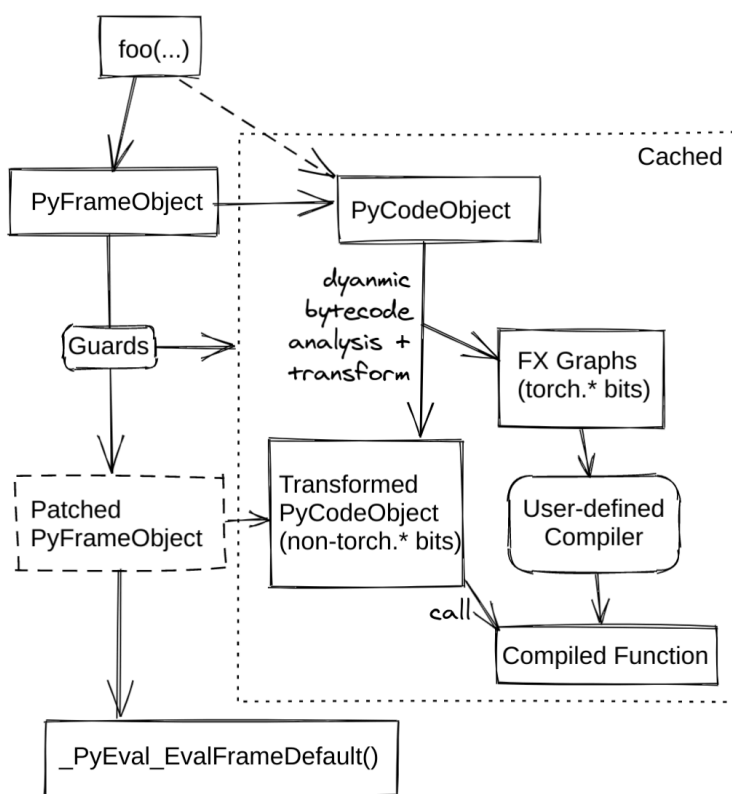
TorchDynamo 是一个 Python JIT 编译器，可以提高 PyTorch 代码的运行速度。那，它是怎么做到的呢？

TorchDynamo 总览

Default Python Behavior



TorchDynamo Behavior



Torch Dynamo Overview

这里本想自己画一个流程图的，不过最后发现 Torch 文档里的这张图本身已经足够说明 TorchDynamo 的工作流程了，所以这里就直接用啦～

对于一个 Python 函数来说，默认当然就是直接用 Python 解释器来执行了，或者更严谨一点来说会将此时的 Frame 会交给 Python 解释器来 Eval。而 [PEP 523](#) 为我们提供了一个修改 Python 解释器 Eval Frame 行为的方法，让我们能够可以自己来解释执行 Frame。TorchDynamo 便是基于此原理来实现的。

对于同样一个 Python 函数，TorchDynamo 会在 Eval Frame 时通过编译将原始的 Frame 转换成一个新的 Frame，两者主要差异在于字节码上，这个新的 Frame 同样会交给 Python 解释器来执行，也就是说 TorchDynamo 的主要工作是在 Eval Frame 初期的字节码变换过程。

图中我们会看到有一个 Guards，它是用来保护 Cache 中直接获取的 CodeObject 的，对于同一个函数的 CodeObject，并不是说编译一次对于之后所有的输入都是可用的，因为 TorchDynamo 的编译过程强依赖于输入类型和值等信息，因此第一次编译后的 CodeObject 可能对于第二次输入是不适用的，Guard 便是用来检查此项的，当 Guard 检查失败时，就会触发重新编译。

TorchDynamo 编译过程

TorchDynamo 在编译过程中会逐字节码进行模拟执行，相当于实现了一个简版的 Python 解释器，在这个过程中会收集所有栈上变量的信息，以及其相关的操作，这也是为什么图中会写「dynamic bytecode analysis」，区别于静态分析，TorchDynamo 可以从 Frame 的 `f_locals` 字段里找到函数的输入，进而将整个函数运行过程模拟出来。

在模拟执行的过程中，如果 TorchDynamo 遇到了 Tensor，就会创建一个 FX Proxy，开始 FX Graph 组网过程，即开始 trace。

在模拟执行结束时，首先会从 FX Graph 生成 Python 函数，并将其挂载到 globals 里。之后在生成的字节码里先将这个函数 LOAD 到栈里，然后将所有输入也 LOAD 到栈上，之后 CALL 这个函数。当然，仅仅凭借如此是无法把全部操作都还原的，对于一些 Python 的 SideEffects，还需要在最后生成相关代码来处理。

这里真正起到加速作用的是 FX Graph，FX Graph 是可以交给各种后端进行编译加速的，比如默认的 TorchInductor 会将 FX Graph 编译到 Triton（GPU）或 C++/OpenMP（CPU）。

Eval Frame 原理和执行流程

FrameObject 和 CodeObject

CodeObject 是指 Python 经过编译后产生的代码对象，它主要包含了 Python 字节码及其相关信息，比如常量表、变量名表等。

FrameObject 是指在函数运行时的栈帧，包含编译时产生的 CodeObject 以及一些运行时的参数信息等。

简单来说，CodeObject 是一个编译时的产物，而 FrameObject 是一个运行时的概念，同一个函数多次运行会产生多个 FrameObject，而其对应的 CodeObject 是同一个。

PEP 523

PEP 523 的标题是 Adding a frame evaluation API to CPython，即为 CPython 添加一个用来 Eval Frame 的 API。这个提案为 JIT 编译提供了可能，允许 JIT 编译器在 Eval Frame 时执行自己的一些操作，比如重新编译原有 CodeObject 生成新的 CodeObject 等。

因此该提案在 PyInterpreterState 上增加了一个 eval_frame 字段，即在 Eval Frame 时会调用的函数。其默认值即是直接调用 Python 解释器默认行为

_PyEval_EvalFrameDefault 函数。而我们可以通过修改它来实现 Eval Frame 行为的自定义，

此外，该提案还在 CodeObject 上添加了一个 co_extra 字段，以便 JIT 编译器在编译时将一些额外的信息存储在 CodeObject 中，比如编译后的 CodeObject 等。

Eval Frame 流程

原理基本都介绍完啦，下面说一下 TorchDynamo 在 Eval Frame 时发生的具体过程。

对于使用 torch.compile 装饰的函数 fn，torch 会生成一个 callback，用于编译 Frame 生成新的 CodeObject，同时这个函数会被装饰成如下：

</>

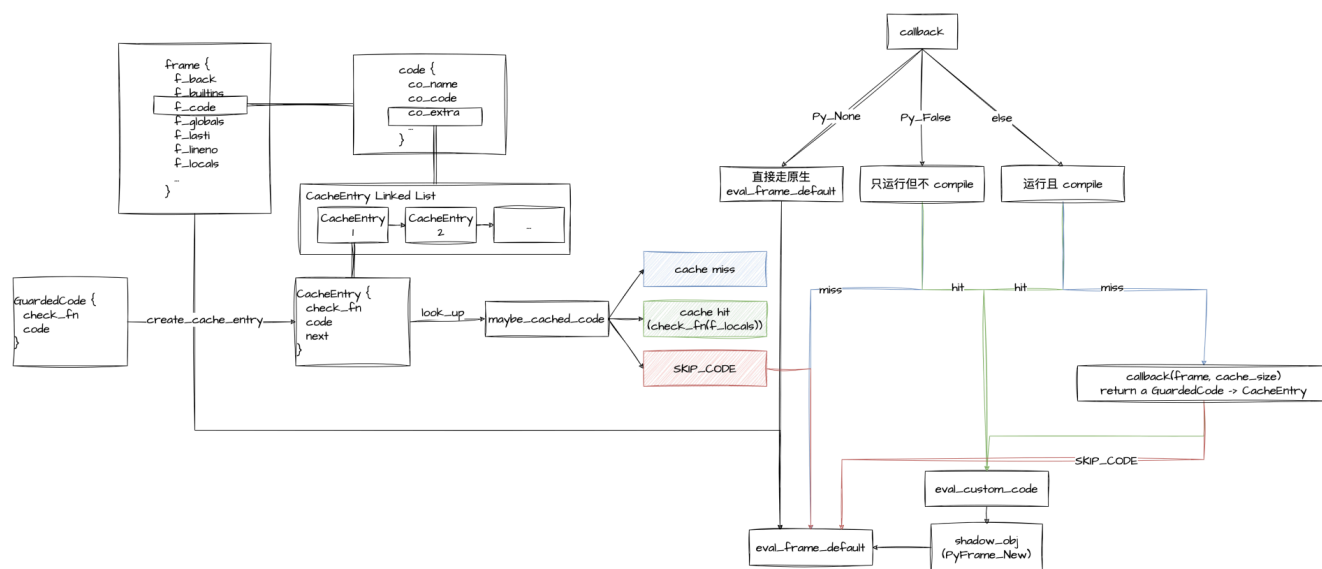
Python | 收起 ^

```

1 @functools.wraps(fn)
2 def _fn(*args, **kwargs):
3     prior = set_eval_frame(callback)
4     try:
5         return fn(*args, **kwargs)
6     finally:
7         set_eval_frame(prior)

```

在调用 `set_eval_frame` 时, Dynamo 会将通过 `tstate->interp->eval_frame = &custom_eval_frame_shim`; 来将 Eval Frame 行为替换成自己的 `custom_eval_frame_shim`。这样之后调用的 `fn` 便是由 Dynamo 自己的 `custom_eval_frame_shim` 来执行的。



Eval Frame 流程

`custom_eval_frame_shim` 的源码分析图如上, 我们看一下它具体是如何工作的。

首先, 对于 Eval Frame 来说, 当然是可以获取到 FrameObject 的, 同样也可以从中获取 CodeObject。根据 PEP 523, 我们可以在 `co_extra` 字段里存储一些 Cache, 这里便会存储 Dynamo 编译后的 CodeObject, 值得注意的是, 这里 `CacheEntry` 是同时包含 `check_fn` 和 `code` 字段的, `check_fn` 即是用于检查一个编译后的 CodeObject 是否可用的, `check_fn` 会作用于 `f_locals` (即 `check_fn(f_locals)`) 来检查该 Cache 是否可用。这主要分为以下三种情况:

- 其中 `CacheEntry` 是一个链表, 默认最大长度为 64, 如果查找全部 Cache 都不可用时, 就会认为是 cache miss;
- 而当 `check_fn(f_locals)` 检查成功时, 就会认为 cache hit;
- 此外还有一种情况, 是 cache 里存储的是 `SKIP_CODE`。

另外, Dynamo 会根据 callback 的情况分别执行如下操作:

- 当 `callback` 为 `Py_None` 时，会直接跑原生字节码，即直接由 `eval_frame_default` 来执行；
- 当 `callback` 为 `Py_False` 时，表示只运行但不 `compile`，即如果 `cache hit` 就跑 `eval_custom_code`，`cache miss` 就跑 `eval_frame_default`
- 当 `callback` 为一个 callable 函数时，表示运行且 `compile`，同样在 `cache hit` 时直接跑 `eval_custom_code`，而当 `cache miss` 时，会先调用 `callback` 编译出新的 `CodeObject`，然后将其存入 `cache`，最后再跑 `eval_custom_code`。

值得注意的是，如果调用 `callback` 返回的是 `None` 时，那么就表示编译失败，此时会将 `cache` 设置为 `SKIP_CODE`，并且直接跑 `eval_frame_default`，而且之后所有的调用都会直接跑 `eval_frame_default`。即只要有一次编译失败，该函数的之后所有的调用都会直接跑原生字节码。不过这明显是合理的，因为对于一个函数而言，编译失败大概率意味着这个函数是不适合编译加速的，那么之后即便是不同的输入也不会再编译了。

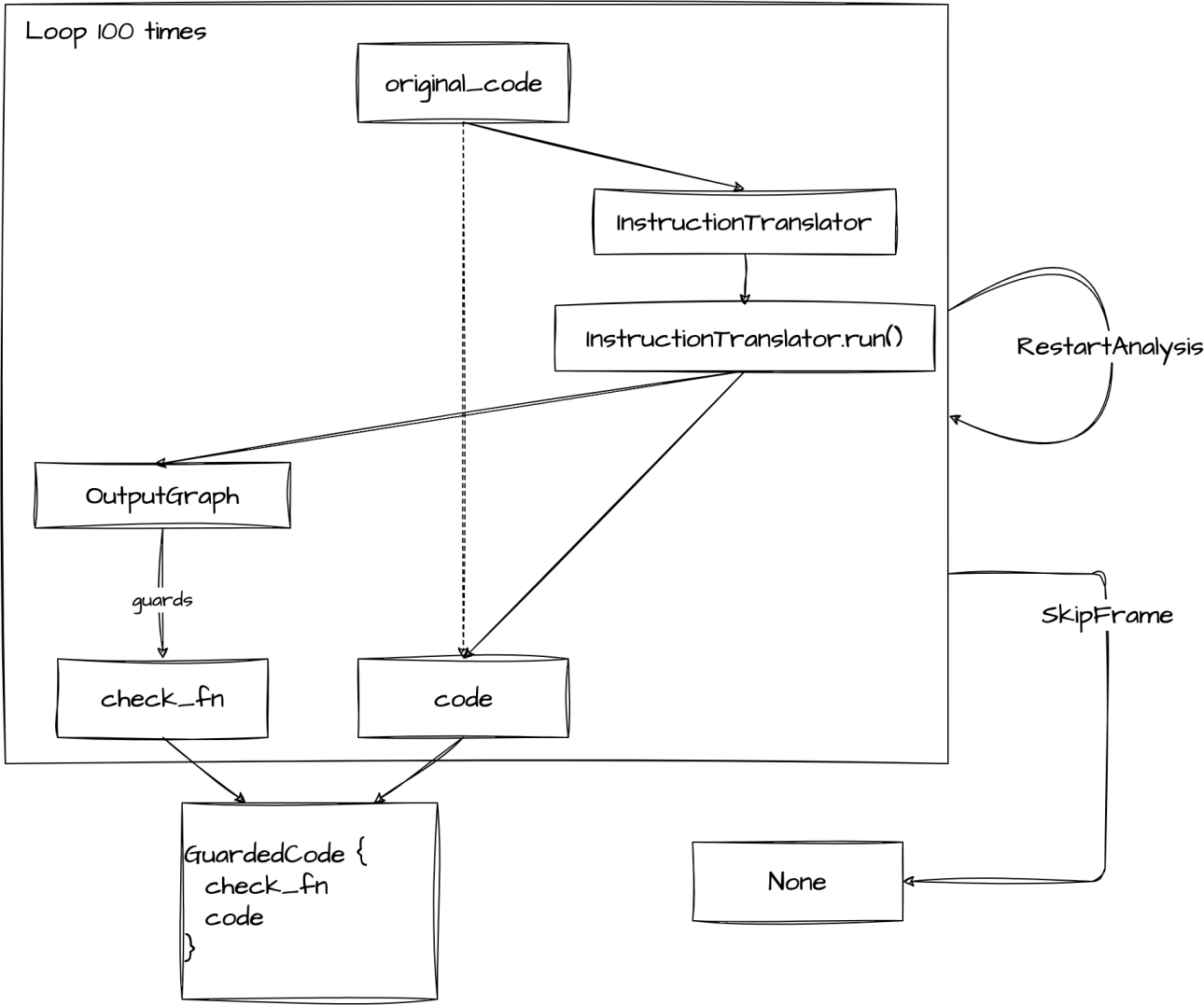
`eval_custom_code` 的实现很简单，就是基于原有 `FrameObject` 创建一个新的，在创建时使用编译后的 `CodeObject` 即可，最后会将新的 `FrameObject` 传入 `eval_frame_default` 来执行。

也就是说 `Dynamo` 的 `Eval Frame` 只会做 `CodeObject` 的转换，最后还是会让 `Python` 解释器来实际执行。

字节码模拟执行

那么在 `callback` 里具体是如何去进行字节码变换的呢？`Dynamo` 是通过模拟执行 + `Codegen` 的方式来实现的。在模拟执行时，`Dynamo` 会收集需要的信息，以在 `Codegen` 时尽可能地还原 `Python` 的行为。

compile 流程



Compile 流程

Dynamo 在 compile 时（也就是 callback）会将逐字节码地模拟执行，在执行过程中如果遇到无法编译的情况，就会抛出 `SkipFrame`，并在 callback 处返回 `None`，以标记该 `CodeObject` 不适合编译。

若模拟执行成功，那么会根据编译得到的 `OutputGraph` 来生成 `check_fn`，与 `Codegen` 得到的 `code` 共同组成 `GuardedCode` 返回给 `Eval Frame`。

模拟执行流程

模拟执行是 Dynamo 的核心，它的所有行为是由 `InstructionTranslator` 来定义的。

为了能够尽可能真实地模拟 Python 解释器的行为，`InstructionTranslator` 在初始化的时候同时初始化了模拟运行栈 `stack`、PC `instruction_pointer`、模拟块栈 `block_stack`、模拟 locals `symbolic_locals`、模拟 globals `symbolic_globals` 等诸多属性，在之后的模拟运行过程中，会不断与这些状态进行交互。

由于是模拟执行，执行过程中是不会对原有数据进行修改的，为了做到这一点，Dynamo 会将所有栈上数据包装成 `VariableTracker`，之后所有的操作都会被 `VariableTracker` 所记录。

`symbolic_locals` 在初始化的时候便会从 `f_locals` 包装成 `VariableTracker`。

</>

Python | 收起 ^

```
1 self.symbolic_locals = collections.OrderedDict(  
2     (  
3         k,  
4         VariableBuilder(  
5             self,  
6             LocalSource(k) if k in code_options["co_varnames"] else  
7             LocalSource((k)),  
8         )(f_locals[k]),  
9     )  
10     for k in vars  
11     if k in f_locals  
12 )
```

因此模拟执行的操作基本都是对 `VariableTracker` 实例及其子类实例的操作，不会影响原始数据。

在调用 `InstructionTranslator.run` 时，Dynamo 会逐步根据字节码 `opname` 分发到对应的函数，比如 `LOAD_CONST` 的实现如下：

</>

Python | 收起 ^

```
1 def LOAD_CONST(self, inst):  
2     # For empty tuples, create empty TupleVariable  
3     if isinstance(inst.argval, tuple) and not inst.argval:  
4         self.push(TupleVariable([]))  
5     else:  
6         self.push(ConstantVariable(value=inst.argval))
```

这里很简单地将包装后的常量压入了模拟运行栈 `stack` 中。

之后看一下稍微复杂一点的 `BINARY_ADD` 的实现：

</>

Python | 收起 ^

```
1 BINARY_ADD = stack_op(operator.add)  
2  
3 def stack_op(fn: typing.Callable[..., object]):  
4     nargs = len(inspect.signature(fn).parameters)  
5     fn_var = BuiltinVariable(fn)  
6  
7     @functools.wraps(fn)  
8     def impl(self: "InstructionTranslatorBase", inst: Instruction):
```

```
9     self.push(fn_var.call_function(self, self.popn(nargs), {}))
10
11     return impl
```

这里先用 builtin 函数（这里是 `operator.add`）创建 `BuiltinVariable`，然后弹栈两个操作数，传入新的 `Variable` 调用 `call_function`，并将结果压栈。

那么这里 `call_function` 做了什么呢？由于这里的实现细节比较多，这里只考虑几种简单的情况：

- 当两个参数都是常量（`ConstantVariable`），并且可以常量折叠，则直接返回折叠后的 `ConstantVariable`
- 如果有参数是 `Tensor`（`TensorVariable`），那么创建 FX Proxy，开始 FX Graph 组网

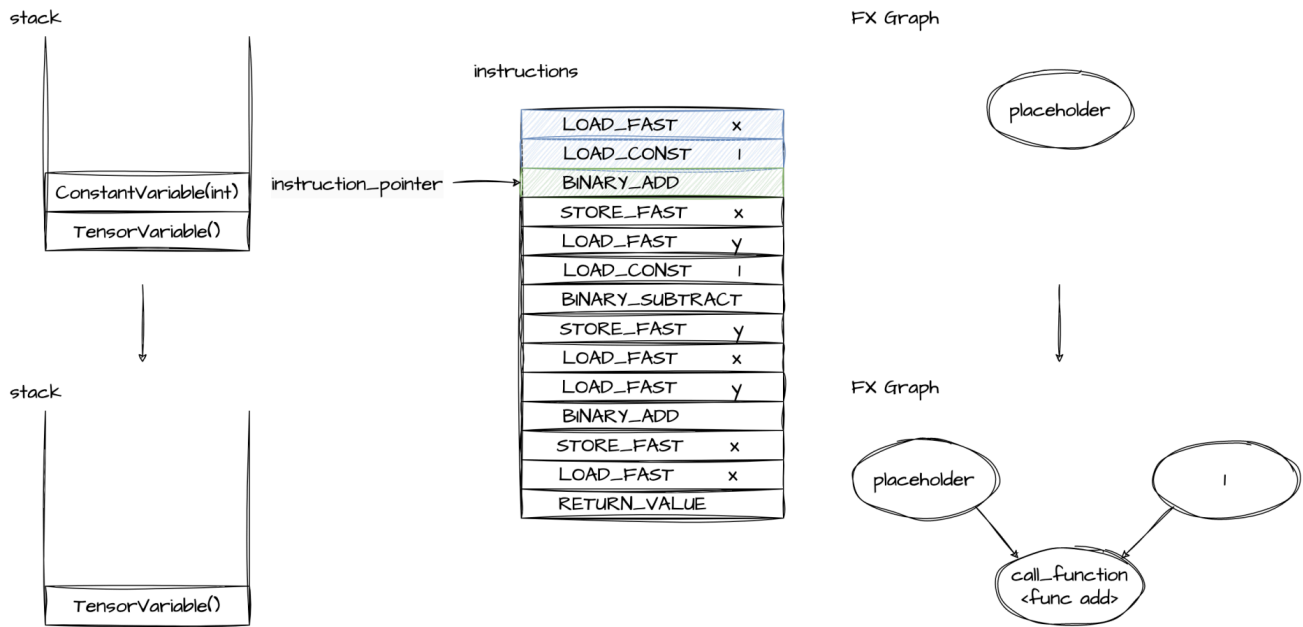
比如对于如下的代码：

</>

Python | 收起 ^

```
1 import torch
2
3
4 @torch.compile
5 def foo(x: torch.Tensor, y: int):
6     x = x + 1
7     y = y - 1
8     x = x + y
9     return x
10
11
12 x = torch.as_tensor(1)
13 y = 2
14 print(foo(x, y))
```

实际模拟执行可能会类似于下图：



模拟执行

在执行 `BINARY_ADD` 前后，`stack` 弹出两个 `Variable`，并放入一个新的 `Variable`，同时 FX Graph 也进行了组网。

由于 `BuiltinVariable` 表示一个 builtin 操作，是有很多操作是会构建此 `Variable` 的，比如各种魔法函数，当然 `print` 也是，但是 `call_function` 时并没有 `print` 的处理方式，因此会抛出 `Unsupported` 异常打断子图。

由于 `BuiltinVariable` 表示一个 builtin 操作，是有很多操作是会构建此 `Variable` 的，比如各种魔法函数，当然 `print` 也是，但是 `call_function` 时并没有 `print` 的处理方式，因此会抛出 `Unsupported` 异常打断子图。

子图打断

在整个代码运行过程中，主要有以下三种情况会打断子图，触发子图编译：

- 当遇到 `RETURN_VALUE` 时
- 当遇到跳转指令时，且跳转条件是 Tensor（`TensorVariable`）时
- 当内部任意时刻抛出 `Unsupported Error` 时

对于遇到 `Unsupported Error` 时，Dynamo 会将当前的子图打断，并将之后的代码抽到一个新的函数中，即交由下一个 Frame 来处理。

跳转指令的处理

跳转指令的处理会稍微复杂一些，Dynamo 会在遇到 `JUMP` 指令，且跳转条件是 Tensor 的时候，触发子图编译，并将跳转分支分别提取成两个函数，分别对原有的两个分支进行替换。

下面直接通过例子来说明一下：

</>

Python | 收起 ^

```
1 @torch.compile
2 def foo(x, y):
3     if x > 0:
4         y += 1
5     else:
6         y -= 1
7     return y
8
9 print(foo(torch.as_tensor([5]), torch.as_tensor([0])))
```

对于如上代码来说，由于跳转条件是一个 Tensor，因此是会发生子图打断的，对于函数 `foo` 而言，编译前后的字节码分别如下：

</> Plain Text | 收起 ^

1 Before:

2

3 42

0 LOAD_FAST

0 (x)

4

2 LOAD_CONST

1 (0)

5

4 COMPARE_OP

4 (>)

6

6 POP_JUMP_IF_FALSE

10 (to 20)

7

8 43

8 LOAD_FAST

1 (y)

9

10 LOAD_CONST

2 (1)

10

12 INPLACE_ADD

11

14 STORE_FAST

1 (y)

12

13 46

16 LOAD_FAST

1 (y)

14

18 RETURN_VALUE

15

16 45

>>

20 LOAD_FAST

1 (y)

17

22 LOAD_CONST

2 (1)

18

24 INPLACE_SUBTRACT

19

26 STORE_FAST

1 (y)

20

21 46

28 LOAD_FAST

1 (y)

22

30 RETURN_VALUE

23

24 After:

25

26 40

0 LOAD_GLOBAL

0 (__compiled_fn_0)

27

2 LOAD_FAST

0 (x)

28

4 CALL_FUNCTION

1

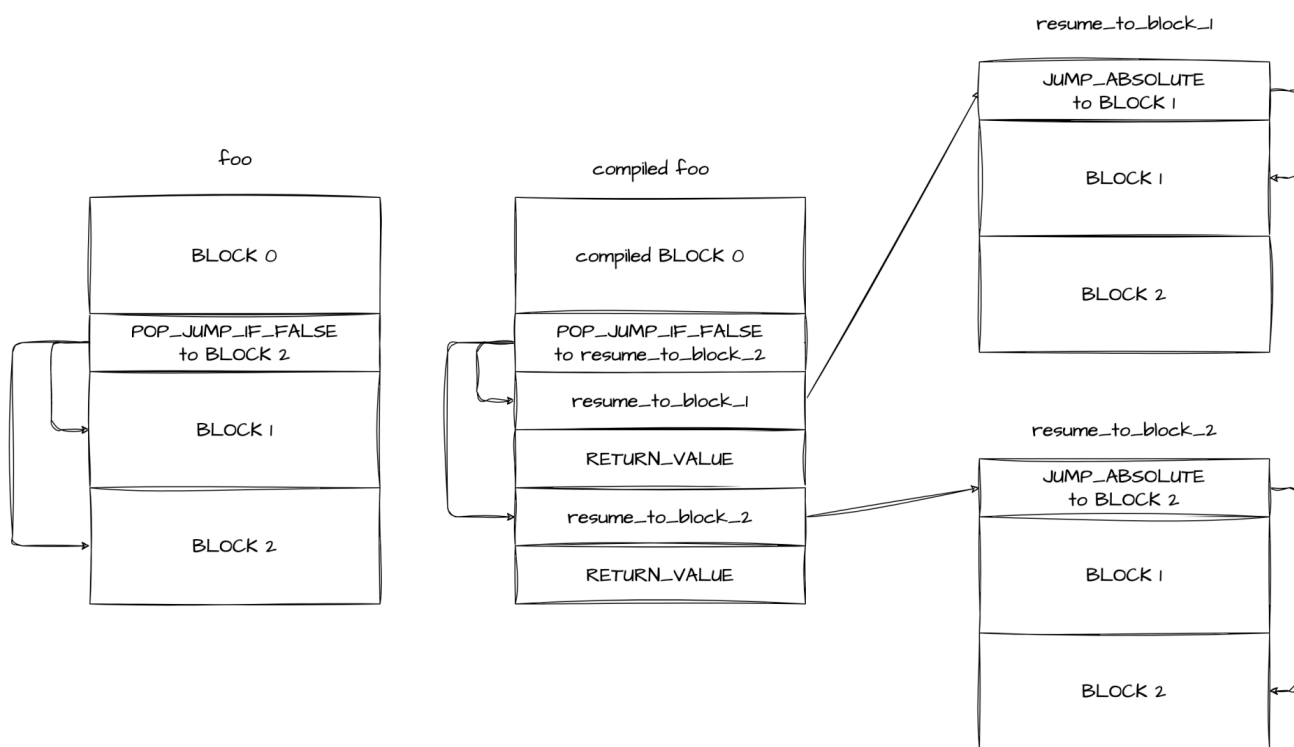
29		6 UNPACK_SEQUENCE	1
30		8 POP_JUMP_IF_FALSE	9 (to 18)
31		10 LOAD_GLOBAL	1 (__resume_at_8_1)
32		12 LOAD_FAST	1 (y)
33		14 CALL_FUNCTION	1
34		16 RETURN_VALUE	
35	>>	18 LOAD_GLOBAL	2 (__resume_at_20_2)
36		20 LOAD_FAST	1 (y)
37		22 CALL_FUNCTION	1
38		24 RETURN_VALUE	
39			
40	__resume_at_8_1:		
41			
42	42	0 JUMP_ABSOLUTE	5 (to 10)
43		2 LOAD_FAST	1 (x)
44		4 LOAD_CONST	1 (0)
45		6 COMPARE_OP	4 (>)
46		8 POP_JUMP_IF_FALSE	11 (to 22)
47			
48	43	>> 10 LOAD_FAST	0 (y)
49		12 LOAD_CONST	2 (1)
50		14 INPLACE_ADD	
51		16 STORE_FAST	0 (y)
52			
53	46	18 LOAD_FAST	0 (y)
54		20 RETURN_VALUE	
55			
56	45	>> 22 LOAD_FAST	0 (y)
57		24 LOAD_CONST	2 (1)
58		26 INPLACE_SUBTRACT	
59		28 STORE_FAST	0 (y)
60			
61	46	30 LOAD_FAST	0 (y)
62		32 RETURN_VALUE	
63			
64	__resume_at_20_2:		
65			
66	42	0 JUMP_ABSOLUTE	11 (to 22)
67		2 LOAD_FAST	1 (x)
68		4 LOAD_CONST	1 (0)
69		6 COMPARE_OP	4 (>)
70		8 POP_JUMP_IF_FALSE	11 (to 22)
71		10 LOAD_FAST	0 (y)

```

72      12 LOAD_CONST          2 (1)
73      14 INPLACE_ADD
74      16 STORE_FAST          0 (y)
75      18 LOAD_FAST           0 (y)
76      20 RETURN_VALUE
77
78 45    >>    22 LOAD_FAST          0 (y)
79          24 LOAD_CONST          2 (1)
80          26 INPLACE_SUBTRACT
81          28 STORE_FAST          0 (y)
82
83 46          30 LOAD_FAST          0 (y)
84          32 RETURN_VALUE
85

```

字节码上可能并不是很清晰，这里用一张图来说明下：



Resume 代码展示

可以看到，生成的字节码中，一方面包含了子图编译的函数，另一方面，将是否跳转对应的两个分支抽取到了新的 resume 函数中，这样在这个函数，根据 Tensor 值来跳转的问题就解决了，下个分支的问题，交由下个 Frame 处理即可，这样问题就分解了。

值得注意的是，在字节码层面，我们不应该去过分的苛求还原原有的分支结构，在这里我们只会注意跳转的与否，如果不跳转，就是走 BLOCK 1 + BLOCK 2，如果跳转，就是走 BLOCK 1，在字节码层面这是很清晰、很明确的事情。这样的话，我们就无需再去考虑它从代码层面是 if-else 还是 for/while-loop，因为在字节码层面他们都是 JUMP，都会归结成这一种模式。那嵌套

的怎么办？嵌套在字节码层面本质就是多个 JUMP 指令，但对于我们来说，只需要关注第一个 JUMP 就可以了，第二个 JUMP 已经移交到下个 Frame 处理了。

代码生成

从端到端来说，整个 Dynamo 做的事就是 `code -> code`，也就是说整个过程一定是一种代码变换，而我们之前模拟执行的主要目的是收集信息，之后还需要 Codegen 来生成需要的代码。

Dynamo 的代码生成部分大多都是非常简单易懂的，resume 部分生成的代码已经在上面展示过了，下面展示下子图编译的代码生成。

子图编译代码生成

子图编译时的代码生成主要包含以下几步：

- 将 FX Graph 编译成函数，这个过程会调用用户提供的 compiler backend，并将该函数挂到 globals 里
- 在生成字节码里加上从 globals 里 LOAD 刚刚编译好的函数的字节码
- 在生成字节码里加上 LOAD 需要传入的参数，注意所有参数都是知道来源（Source）的，因此可以 Codegen 出需要的 LOAD 指令
- 在生成字节码里加上 CALL_FUNCTION 指令，调用编译好的函数
- 在生成字节码里加上 SideEffects 的处理

我们从一个示例来看这个过程：

</>

Python | 收起 ^

```
1 class A:
2     def __init__(self, value):
3         self.value = value
4
5 @torch.compile
6 def foo(x, y):
7     x.value += 1
8     return x.value + y.value
9
10 foo(A(3), A(torch.Tensor([4])))
```

生成的代码大致如下（因为生成的代码是字节码层面的，这里手动还原成 Python 代码）：

</>

Python | 收起 ^

```
1 # 这个函数只有字节码，没有 Python 源码，这是手动翻译过来的
2 def foo(x, y):
3     out = compiled_fn(y.value)
```

```
4     out[0] # 这里直接放在栈上没有取走，所以最后 return 的时候会 return 这个，这里
           尽可能还原字节码顺序，不把这个放在 return 后面
5     x.value = 4 # SideEffects 生成的
6     return # out[0]
7
8 # 这个函数是 FX graph 生成的
9 def compiled_fn(y_value):
10     add = 4 + y_value
11     return (add, )
```

值得注意的是，这里两个函数里包含了常量 4，这个 4 是 `x.value += 1` 生成的，也就是说这个输入已经硬编码在输出的代码里了，为了保证其正确性，Dynamo 在 Guard 里包含了 `L['x'].value == 3` 的检查项，也就是说只要 `x.value != 3` 就会触发重新编译。

Source 管理

在上面的代码生成过程中，我们提到了 Source，我们可以利用它来重新生成 LOAD 参数的代码，这里看一下 Source 是如何管理的。

在模拟执行的初期，我们从 `f_locals` 构建 `symbolic_locals` 的时候，就会设置生成的 `VariableTracker`，使其 Source 为 `LocalSource`，这样就可以知道这些栈上元素最初是从 `f_locals` 里得到的，当然我们就可以生成直接从 `f_locals` 里 LOAD 它的指令了。

对于一些复合操作，比如 `x.y`，在 `VariableTracker` 的构建过程中，会将 Source 属性进行传播，比如这里 `x.y` 对应的 `VariableTracker` 的 Source 可能就是 `AttrSource(base=LocalSource(local_name='x'), member='y')`，我们同样也是可以很方便地通过它来生成这个 `getattr` 操作对应的 Bytecode。

Guard 管理

为了保护生成的代码在下一次调用时是可用的，Dynamo 会为每个生成的代码都加上一个 Guard，这个 Guard 是一个 lambda 函数，形式大概类似 `lambda L:`

```
__guarded_code.valid and not
__are_deterministic_algorithms_enabled() and __check_tensors(L['x'],
L['y'])
```

在 Eval Frame 部分我们已经知道了，这个函数会将 `f_locals` 作为参数，因此，这里的 `L['x']`、`L['y']` 即表示输入参数中的 `x` 和 `y`。

比如对于如下代码：

</>

Python | 收起 ^

```
1 class A:
2     def __init__(self, value):
3         self.value = value
4
```

```
5
6 @torch.compile
7 def foo(x, y):
8     if x.value > 3:
9         y += 1
10    else:
11        y -= 1
12    return y
13
14 print(foo(A(5), torch.as_tensor([0])))
```

最后会生成如下的 Guard:

```
</> Python | 收起 ^

1 lambda L: ___guarded_code.valid and ___check_type_id(L['x'], 4882921328)
  and ___check_type_id(L['x'].value, 4299954872) and L['x'].value == 5 and
  not ___are_deterministic_algorithms_enabled() and
  ___check_tensors(L['y'])
```

这里主要检查了如下几项:

- `x` 类型是 `A`
- `x.value` 类型是 `int` , 这里 `x.value` 的代码同样可以从 Source 生成
- `x.value` 值为 5
- 关于 `y` 的一系列 Tensor 检查, 这包含了 `dtype` 、 `device` 、 `requires_grad` 、 `ndim` 等属性

Guard 是有传播机制的, 比如 `z = x + y` , 新生成的 `z` 的 Guard 是 `x` 和 `y` 是 Guard 的总和, 这可以通过 `VariableTracker.propagate` 在不同 `VariableTracker` 之间进行传播, 这就确保了值的依赖关系是可以保持的。

SideEffects 管理

由于在模拟执行过程中, 我们是可以跟踪任意时刻对于任何变量的操作的, 因此任何副作用都是可追踪的, 在遇到存在副作用的操作时, 我们只需要将其记录下来, 并在最终生成的代码里生成相应的副作用代码即可。

比如对于如下明显有副作用的代码:

```
</> Python | 收起 ^

1 @torch.compile
2 def foo(x):
3     x.append(1)
4     return x[0] + 1
```

```
5
6
7 x = [torch.Tensor([1])]
8 print(foo(x))
```

生成的代码大概如下：

</>

Python | 收起 ^

```
1 def foo(x):
2     out = compiled_fn(x[0])
3     out[0]
4     x[:] = [x[0], 1] # SideEffects
5     return # out[0]
6
7 def compiled_fn(x_0):
8     add = x_0 + 1
9     return (add, )
```

其中 `x[:] = [x[0], 1]` 即恢复副作用影响的代码，Dynamo 会将原来的 `x` inplace 地全部替换成新的结果 `[x[0], 1]`

References

1. [PEP 523 – Adding a frame evaluation API to CPython](#)
2. [TorchDynamo Overview - Guards Overview](#)
3. [深入理解 Python 虚拟机](#)